

Chapter 27: Dynamic Components

1. Overview

Every Angular app starts with static, declarative templates: you write `<app-child>` in a parent's HTML, the compiler wires it up, and the framework instantiates it whenever the parent view renders. That model covers the vast majority of UI, but it breaks down the moment the *type* of component you need to render is only known at runtime — a modal service that can host any component the caller passes in, a plugin system that loads widgets by name from a registry, a dynamic form renderer that builds a tree of field components from a JSON schema, or a notification/toast stack that stamps out a new component instance per message.

For all of these, you need **dynamic component creation**: imperatively instantiating a component class, attaching it to the DOM/view tree, feeding it inputs, listening to its outputs, and eventually tearing it down — all through TypeScript code rather than template markup.

The Angular API surface for this has changed meaningfully across major versions:

- Pre-Ivy (View Engine, Angular ≤8): you needed `ComponentFactoryResolver.resolveComponentFactory(Component)` to get a `ComponentFactory`, then `viewContainerRef.createComponent(factory)`.
- Ivy (Angular 9+): `ComponentFactory` became a thin wrapper; `ComponentFactoryResolver` was deprecated but kept for compatibility.
- Angular 13+: `ViewContainerRef.createComponent()` gained an overload that takes the **component class directly** — no factory, no resolver needed at all. `ComponentFactoryResolver` was formally deprecated for removal.
- Angular 14+: `createComponent` (the standalone function from `@angular/core`, used for bootstrapping without a host view) and `ComponentRef.setInput()` arrived, making input-setting through change detection safe.
- Angular 19: `ComponentFactoryResolver` was removed entirely from the public API in the deprecated form (the old factory-based path is gone from modern docs; `NgModuleFactory`-adjacent APIs continue to be pared down).

This chapter covers the full modern (v13+) API for dynamic components:

`ViewContainerRef.createComponent`, `ComponentRef`, `NgComponentOutlet`, `NgTemplateOutlet`, building a real dynamic modal/dialog service, and the memory-leak and change-detection pitfalls that show up constantly in production code and in interviews.

2. Core Concepts

2.1 Why templates alone aren't enough

A template like:

```
@if (showChild) {  
  <app-child [data]="data"></app-child>  
}
```

still requires `AppChildComponent` to be **known at compile time** — it must be imported and referenced (directly or via a standalone `imports` array). This works when you have a fixed, small set of possible components. It does not work when:

- The component type is decided by data (a "widget type" string from an API response).
- You're building a library that must render *host-application* components it has never seen (a generic modal/dialog service, a toast library, a grid cell renderer, a plugin host).
- You need N instances of something on demand, not driven by `*ngFor` over a stable array (e.g., "spawn a new tooltip/popover each time the user hovers").

In these cases you reach for `ViewContainerRef`, which is the runtime handle Angular gives you to imperatively insert views (embedded or component) at a specific location in the view hierarchy.

2.2 ViewContainerRef

A `ViewContainerRef` represents an anchor point in the DOM/view tree where views can be added as siblings, in order. You get one by:

```
@ViewChild('anchor', { read: ViewContainerRef }) anchor!: ViewContainerRef;
```

or by injecting it in a directive/component constructor (it resolves to the container attached to that element):

```
constructor(private vcr: ViewContainerRef) {}
```

or via a template reference combined with `{ read: ViewContainerRef }`, or through an `ng-template` + structural-directive-style host.

Key methods:

Method	Purpose
<code>createComponent(componentType, options?)</code>	Instantiate a component and insert its host view into this container
<code>createEmbeddedView(templateRef, context?)</code>	Instantiate an <code>ng-template</code> and insert it
<code>insert(viewRef, index?)</code>	Move/re-insert an existing view
<code>remove(index?)</code>	Destroy and remove a view at index
<code>clear()</code>	Destroy and remove all views in the container
<code>indexOf(viewRef) / get(index) / length</code>	Introspection
<code>element</code>	The DOM element this container is anchored to

2.3 ViewContainerRef.createComponent() — modern signature (v13+)

```
createComponent<C>(  
  component: Type<C>,  
  options?: {  
    index?: number;  
    injector?: Injector;  
    ngModuleRef?: NgModuleRef<unknown>;  
    projectableNodes?: Node[][];  
  }  
) : ComponentRef<C>
```

- `component` — the component **class**, imported directly. No decorator gymnastics, no factory resolution.
- `index` — where in the container to insert it (defaults to the end).
- `injector` — a custom injector so the dynamic component can receive DI tokens that aren't available from the ambient injector (critical for standalone dynamic modals — see §3).
- `projectableNodes` — arrays of DOM nodes to project into the component's `<ng-content>` slots, positionally matched to `ngContentSelectors`.

For **standalone components** (default since Angular 19; explicit `standalone: true` in 14–18), no `NgModule` import or `entryComponents` declaration is needed — that whole legacy concept (`entryComponents`, `ANALYZE_FOR_ENTRY_COMPONENTS`) disappeared with Ivy, because Ivy compiles every component so it can always be looked up and instantiated, whether or not it's referenced in a template.

2.4 ComponentRef<C>

`createComponent` returns a `ComponentRef<C>`, your handle to the live instance:

```
interface ComponentRef<C> {  
  location: ElementRef;           // host DOM element  
  injector: Injector;            // the component's own injector  
  instance: C;                   // the actual component instance – set @Input()-bound  
  // properties here  
  hostView: ViewRef;             // the view containing this component  
  componentType: Type<C>;  
  changeDetectorRef: ChangeDetectorRef;  
  setInput(name: string, value: unknown): void; // v14+ – safe input setting  
  destroy(): void;  
  onDestroy(callback: () => void): void;  
}
```

- `instance` — direct property access to set `@Input()` s or call public methods, and to `.subscribe()` to `@Output()` `EventEmitter` s.
- `setInput(name, value)` (Angular 14+) — the *preferred* way to set inputs. Unlike `instance.prop = value`, it:
 - Marks the input as "dirty" so Angular's change detection and `ngonchanges` lifecycle hook fire

correctly.

- Works even before the first change detection run.
- Is required if the target uses **signal inputs** (`input()`) — direct assignment to a signal-backed input doesn't work the same way; `setInput` is the API-stable path regardless of whether the target declares decorator inputs or signal inputs.
- `hostView` — a `ViewRef/EmbeddedViewRef`-like object representing the component's view; use it to manually trigger `detectChanges()`, or pass to `ApplicationRef.attachView()` if you built the component completely outside any container (see §4).
- `destroy()` — synchronously runs `ngOnDestroy` on the instance, removes the DOM node, and detaches the view from change detection. **This is the #1 thing people forget**, causing the primary class of dynamic-component memory leaks (§5).

2.5 Two ways to create dynamic components

A. Inside a `ViewContainerRef` (most common)

The new component's host view becomes a sibling view inside the container, and Angular's change detection tree automatically walks into it during the normal top-down check, because the container is already part of the view tree.

```
const ref = this.vcr.createComponent(WidgetComponent);
ref.setInput('title', 'Hello');
```

B. Standalone, unattached (the `createComponent` free function from `@angular/core`, v14+)

```
import { createComponent, EnvironmentInjector, ApplicationRef } from '@angular/core';

const compRef = createComponent(WidgetComponent, {
  environmentInjector: this.envInjector,
});
this.appRef.attachView(compRef.hostView);
document.body.appendChild(compRef.location.nativeElement);
```

This is how Angular Material's `overlay`/CDK portals and Angular's own `bootstrapApplication` internals work — the component isn't inside any existing container, so you must manually `attachView()` it to `ApplicationRef` so it participates in global change detection, and manually append its DOM node wherever you want it (commonly `document.body` for overlays/modals so they escape any `overflow: hidden` ancestor).

2.6 `NgComponentOutlet`

`NgComponentOutlet` is a structural directive that lets you declare "render whatever component type this expression evaluates to" **directly in a template**, without manually touching `ViewContainerRef`:

```

<ng-container *ngComponentOutlet="
  currentComponent;
  inputs: componentInputs;
  injector: customInjector;
  content: projectedContent
"></ng-container>

```

- `currentComponent: Type<any> | null` — the component class to render; changing it destroys the old instance and creates a new one.
- `inputs` (Angular 17.1+) — an object map of input bindings, reactively re-applied via `setInput` whenever the object reference or its content changes (Angular diffs it each CD cycle).
- `injector` — optional custom injector for the created component.
- `content` — optional `Node[][]` for content projection, same shape as `projectableNodes`.

Under the hood, `NgComponentOutlet` is a thin, well-tested wrapper around exactly the `viewContainerRef.createComponent + setInput + .clear()` dance you'd otherwise write by hand — reach for it whenever the "outlet" lives inside a template you control, and reach for manual `viewContainerRef` code when you need imperative control (a service creating a component with no fixed template anchor, e.g., a modal service).

2.7 NgTemplateOutlet

`NgTemplateOutlet` renders an `ng-template`'s content (an embedded view, not a component) at a given spot, optionally with a **context object** the template can destructure via `let-x` bindings.

```

<ng-template #row let-item let-i="index" let-isLast="last">
  <div>{{ i }}: {{ item.name }} {{ isLast ? '(last)' : '' }}</div>
</ng-template>

<ng-container
  *ngTemplateOutlet="row; context: { $implicit: item, index: i, last: isLast }">
</ng-container>

```

- `$implicit` maps to the unqualified `let-item`.
- Named context keys map to `let-i="index"`.
- This is exactly the mechanism `*ngFor`, `*ngIf else`, and custom structural directives use internally — `*ngFor` creates one embedded view per item via `createEmbeddedView(templateRef, { $implicit: item, index, ... })`, which is functionally identical to what `NgTemplateOutlet` exposes to userland.
- Useful for **render-prop style** components: a `<app-list>` that accepts an `itemTemplate: TemplateRef<any> @Input()` from its consumer and outlets it per row, letting the parent fully control row markup while the list component controls the loop/virtualization.

2.8 Injector context for dynamic components

Every component has an injector that, by default, is a child of the injector at its creation site (the `viewContainerRef`'s injector, which is usually the enclosing component's injector, which chains up to the module/environment injector). Dynamically created components follow the *exact same DI rules* as static ones: they can inject anything visible from that chain, plus whatever you pass in `options.injector` — commonly used to inject per-instance **data tokens** into a dynamically created modal (see §3, `MODAL_DATA`).

2.9 Dynamic forms and plugin architectures (use cases)

- **Dynamic forms:** a JSON schema (`{ type: 'text', label: 'Name' }[]`) is mapped to a registry of field components (`TextFieldComponent`, `SelectFieldComponent`, ...), and a loop creates one dynamic component per field via `viewContainerRef.createComponent`, wiring each to a shared `FormGroup / AbstractControl` via inputs.
- **Plugin architecture:** a host app defines an extension point (a `ViewContainerRef` anchor) and a plugin registry maps string keys to lazy-loaded component classes (often behind dynamic `import()` for code-splitting); at runtime the host resolves the key, dynamically imports the chunk, and calls `createComponent` on whatever class the chunk exports — this is the backbone of micro-frontend and "widget marketplace" style apps in Angular.
- **Dynamic modal/dialog/toast services:** covered in full below.

3. Code Examples

3.1 A full dynamic modal service

```
// modal.tokens.ts
import { InjectionToken } from '@angular/core';

export const MODAL_DATA = new InjectionToken<unknown>('MODAL_DATA');

export interface ModalRef<R = unknown> {
  close(result?: R): void;
  afterClosed(): Observable<R | undefined>;
}

export const MODAL_REF = new InjectionToken<ModalRef>('MODAL_REF');
```

```
// modal.service.ts
import {
  Injectable,
  ApplicationRef,
  EnvironmentInjector,
  Injector,
  ComponentRef,
  Type,
  createComponent,
} from '@angular/core';
import { Subject, Observable } from 'rxjs';
```

```

import { MODAL_DATA, MODAL_REF, ModalRef } from './modal.tokens';

export interface ModalConfig<D = unknown> {
  data?: D;
  panelClass?: string;
}

@Injectable({ providedIn: 'root' })
export class ModalService {
  private readonly appRef = inject(ApplicationRef);
  private readonly envInjector = inject(EnvironmentInjector);
  private readonly parentInjector = inject(Injector);

  open<C, R = unknown, D = unknown>(
    component: Type<C>,
    config: ModalConfig<D> = {},
  ): ModalRef<R> {
    const afterClosed$ = new Subject<R | undefined>();

    // Overlay backdrop, created the same way as the content component.
    const backdropEl = document.createElement('div');
    backdropEl.className = 'modal-backdrop';
    document.body.appendChild(backdropEl);

    const modalRef: ModalRef<R> = {
      close: (result?: R) => {
        afterClosed$.next(result);
        afterClosed$.complete();
        componentRef.destroy(); // 1. runs ngOnDestroy on the modal content
        backdropEl.remove(); // 2. clean up the manually-appended DOM
      },
      afterClosed: (): Observable<R | undefined> => afterClosed$.asObservable(),
    };

    // Per-instance injector: lets the dynamic component `inject(MODAL_DATA)`
    // and `inject(MODAL_REF)` just like any normal DI token.
    const modalInjector = Injector.create({
      parent: this.parentInjector,
      providers: [
        { provide: MODAL_DATA, useValue: config.data },
        { provide: MODAL_REF, useValue: modalRef },
      ],
    });

    // createComponent (free function) – not attached to any existing
    // ViewContainerRef, because a modal must escape the component tree
    // (avoid ancestor `overflow:hidden`, z-index stacking contexts, etc.)
    const componentRef: ComponentRef<C> = createComponent(component, {
      environmentInjector: this.envInjector,
      elementInjector: modalInjector,
    });

    if (config.panelClass) {

```

```

    (componentRef.location.nativeElement as HTMLElement).classList.add(config.panelClass);
  }

  // Wire it into Angular's change detection and the real DOM.
  this.appRef.attachView(componentRef.hostView);
  document.body.appendChild(componentRef.location.nativeElement);

  // If the DOM node is torn down through some other path, make sure
  // we still detach from ApplicationRef to avoid a dangling view.
  componentRef.onDestroy(() => this.appRef.detachView(componentRef.hostView));

  return modalRef;
}
}

```

```

// confirm-dialog.component.ts – an arbitrary consumer component, knows nothing
// about ModalService except the two tokens it may choose to inject.
import { Component, inject } from '@angular/core';
import { MODAL_DATA, MODAL_REF, ModalRef } from '../modal.tokens';

@Component({
  selector: 'app-confirm-dialog',
  standalone: true,
  template: `
    <div class="dialog">
      <p>{{ data.message }}</p>
      <button (click)="ref.close(true)">Yes</button>
      <button (click)="ref.close(false)">No</button>
    </div>
  `,
})
export class ConfirmDialogComponent {
  data = inject<{ message: string }>(MODAL_DATA);
  ref = inject<ModalRef<boolean>>(MODAL_REF);
}

```

```

// usage
const ref = this.modalService.open<ConfirmDialogComponent, boolean, { message: string }>(
  ConfirmDialogComponent,
  { data: { message: 'Delete this record?' } },
);

ref.afterClosed().subscribe(confirmed => {
  if (confirmed) this.deleteRecord();
});

```

3.2 The same modal, built via `ViewContainerRef` instead (host-anchored variant)

Useful when the modal only needs to render inside a fixed "portal outlet" already in the app shell, rather than being appended straight to `document.body`.


```

@Component({
  selector: 'app-root',
  standalone: true,
  template: `<ng-template #portalOutlet></ng-template>`,
})
export class AppComponent {
  @ViewChild('portalOutlet', { read: ViewContainerRef, static: true })
  portalOutlet!: ViewContainerRef;
}

```

```

open<C>(component: Type<C>, injector?: Injector): ComponentRef<C> {
  // ViewContainerRef.createComponent automatically attaches the new
  // host view into the container's own view tree – no manual
  // ApplicationRef.attachView() needed, because it's already inside
  // a view that Angular is tracking.
  return this.portalOutlet.createComponent(component, { injector });
}

```

3.3 NgComponentOutlet — data-driven widget rendering

```

@Component({
  standalone: true,
  imports: [NgComponentOutlet],
  template: `
    <ng-container
      *ngComponentOutlet="widgetType; inputs: widgetInputs; injector: widgetInjector">
    </ng-container>
  `,
})
export class DashboardTileComponent {
  @Input() widgetType!: Type<unknown>;
  @Input() widgetInputs: Record<string, unknown> = {};
  widgetInjector = inject(Injector);
}

```

```

// Registry mapping API-provided string keys to component classes:
const WIDGET_REGISTRY: Record<string, Type<unknown>> = {
  chart: ChartWidgetComponent,
  table: TableWidgetComponent,
  kpi: KpiWidgetComponent,
};

// In the parent:
tiles = this.dashboardApi.tiles.map(t => ({
  type: WIDGET_REGISTRY[t.kind],
  inputs: t.config,
}));

```

```
@for (tile of tiles; track tile) {
  <app-dashboard-tile [widgetType]="tile.type" [widgetInputs]="tile.inputs" />
}
```

3.4 NgTemplateOutlet with context — render-prop list

```
@Component({
  standalone: true,
  imports: [NgTemplateOutlet],
  selector: 'app-generic-list',
  template: `
    @for (item of items; track item.id; let i = $index, isLast = $last) {
      <ng-container
        *ngTemplateOutlet="rowTemplate; context: { $implicit: item, index: i, last: isLast
      }">
    </ng-container>
    }
  `,
})
export class GenericListComponent<T> {
  @Input() items: T[] = [];
  @Input() rowTemplate!: TemplateRef<{ $implicit: T; index: number; last: boolean }>;
}
```

```
<app-generic-list [items]="users" [rowTemplate]="userRow"></app-generic-list>

<ng-template #userRow let-user let-i="index" let-isLast="last">
  <div class="row" [class.last]="isLast">{{ i }}. {{ user.name }}</div>
</ng-template>
```

3.5 Dynamic form fields (plugin-style registry)

```
const FIELD_COMPONENTS: Record<FieldType, Type<DynamicField>> = {
  text: TextFieldComponent,
  select: SelectFieldComponent,
  checkbox: CheckboxFieldComponent,
};

interface DynamicField {
  control: FormControl;
  config: FieldConfig;
}

@Component({
  standalone: true,
  selector: 'app-dynamic-form',
  template: `<ng-template #anchor></ng-template>`,
})
export class DynamicFormComponent implements OnChanges, OnDestroy {
  @Input() schema: FieldConfig[] = [];
```

```

@Input() form!: FormGroup;
@ViewChild('anchor', { read: ViewContainerRef, static: true }) anchor!: ViewContainerRef;

private refs: ComponentRef<DynamicField>[] = [];

ngOnChanges(): void {
  this.rebuild();
}

private rebuild(): void {
  this.anchor.clear(); // destroys every previously created field component
  this.refs = [];

  for (const config of this.schema) {
    const ref = this.anchor.createComponent(FIELD_COMPONENTS[config.type]);
    ref.setInput('config', config);
    ref.setInput('control', this.form.get(config.key) as FormControl);
    this.refs.push(ref);
  }
}

ngOnDestroy(): void {
  // vcr.clear() on the parent's destroy already tears these down,
  // but being explicit documents intent and is safe if `anchor`
  // ever changes to a container we don't fully own.
  this.refs.forEach(r => r.destroy());
}
}

```

4. Internal Working

4.1 What "outside the declared template tree" really means

A statically-declared child (`<app-child>` in a parent's template) is emitted by the Angular compiler as part of the parent's **component definition** (`ɵcmp`) — specifically its `template` function, which contains an `elementStart / element` instruction for the child's host element and a directive/component instantiation instruction bound at a fixed index in the parent's `LView`. The child's existence is baked into the compiled output; there is no runtime decision involved.

A dynamically created component has no such instruction in anyone's compiled template. Instead, `ViewContainerRef.createComponent()`:

1. Looks up the component's **definition** (`ɵcmp`) directly off the class (every Ivy-compiled component carries its own definition statically as `MyComponent.ɵcmp`) — no resolver, no registry lookup, no factory needed, because Ivy compiles metadata onto the class itself (locality principle).
2. Creates a new `LView` for the component using that definition — a plain array-based data structure holding the component's DOM nodes, bindings, and child view references, structurally identical to the `LView` any statically-created component would get.
3. Wraps that `LView` into a `ComponentRef`, and creates a new **host view** (`ViewRef`) around it.

4. Inserts the host element into the DOM at the container's anchor position (a comment node, `<!-- container-->`, marks where the container's children begin) and inserts the new view into the container's internal view list at the given `index` (or the end).
5. If created through a `ViewContainerRef` that is *already* attached to a view tree Angular is tracking (i.e., any container obtained from an existing directive/component or an already-attached `ng-template`), the new view is spliced directly into that tree — no manual "attach" step required, because the container itself is already reachable from `ApplicationRef` through its ancestors.
6. If created through the free `createComponent()` function with no container (§2.5B), the resulting `ComponentRef.hostView` is a fully standalone `ViewRef` not reachable from anything — you must call `ApplicationRef.attachView()` yourself, or it will never be checked by change detection and will never be destroyed automatically when its "owner" is destroyed (there is no owner).

4.2 Its own injector

Every `LView` carries a reference to the injector that was active when it was created. For a component made via `createComponent`, Angular builds a `NodeInjector` for the new host element, whose parent is either:

- the `injector` option you pass explicitly, or
- the injector of the `ViewContainerRef` (or, for the free function, the `elementInjector` / `environmentInjector` you supply), which itself typically chains to the injector of the component that owns that `ViewContainerRef`.

This is why the modal service pattern in §3.1 works: `Injector.create({ parent, providers: [...] })` builds a small injector layer containing `MODAL_DATA` / `MODAL_REF`, and because it's passed as the *parent* for the dynamically created component's own injector, `inject(MODAL_DATA)` inside the modal content resolves correctly — DI resolution walks up exactly the same parent chain any static component would use, it's simply a chain you assembled by hand instead of one the compiler assembled from the template's ancestry.

4.3 Hooking into the host view's change detection

Angular's change detection walks the **view tree**, not the "component tree" as drawn in a diagram — a parent `LView` holds references to its child `LViews` (via `TView.childs` / embedded view lists on containers), and `ApplicationRef.tick()` walks from every root view downward, calling `refreshView()` recursively.

- If the dynamic component's host view was inserted into a `ViewContainerRef` that belongs to some ancestor view already in that tree, the new view is now a child of that tree, so it participates in the *very next* `ApplicationRef.tick()` (typically triggered by zone.js after any async task, or manually with `ApplicationRef.tick()` / `ChangeDetectorRef.detectChanges()` in zoneless apps) — exactly like a statically declared child. No extra step needed.
- If it was created via the standalone `createComponent()` free function with no container, its `hostView` is disconnected from every root, so it is **never** checked automatically. `ApplicationRef.attachView(componentRef.hostView)` is what registers it as one of `ApplicationRef`'s root views (pushed into `ApplicationRef['_views']`), making it subject to `tick()` from then on. Forgetting this call is a common bug: the component renders once (its first CD run happens during creation/initial render in some code paths) but never updates again, and worse, if you later expect `appRef.detachView` to run automatically on destroy, it does not — you must call it yourself (as shown in §3.1's `onDestroy` hook) or `ApplicationRef` keeps a reference to a destroyed view forever, itself a leak.

- `ComponentRef.changeDetectorRef` gives you a scoped `ChangeDetectorRef` for just that component's subtree if you need to `detectChanges()` in isolation (e.g., inside a `zoneless / onPush`-heavy dynamic-forms scenario where you set several inputs imperatively and want a single synchronous refresh rather than waiting for the next tick).

4.4 `ComponentFactoryResolver` deprecation/removal timeline

Version	State
≤ Angular 8 (View Engine)	Only path: <code>resolver.resolveComponentFactory(Comp) → ComponentFactory → viewContainerRef.createComponent(factory)</code> . Components had to be listed in <code>entryComponents</code> (NgModule) if not referenced in any template, so the compiler would generate a factory for them.
Angular 9–12 (Ivy)	<code>ComponentFactory</code> still exists but is largely a thin, mostly-unnecessary wrapper — Ivy no longer needs <code>entryComponents</code> at all (removed from <code>@NgModule</code> requirements) because every compiled component already carries an internal definition (<code>@cmp</code>) referenceable directly. <code>ComponentFactoryResolver</code> still works but does extra work for nothing.
Angular 13	<code>viewContainerRef.createComponent(Type<C>, options)</code> overload ships — first version where you can skip <code>ComponentFactoryResolver</code> entirely and pass the class straight in. <code>ComponentFactoryResolver</code> is marked <code>@deprecated</code> .
Angular 14	The standalone <code>createComponent()</code> function and <code>ComponentRef.setInput()</code> land, completing the factory-free imperative API surface, including for standalone components.
Angular 15–18	<code>ComponentFactoryResolver</code> remains as a deprecated, no-op-ish shim purely for back-compat with old libraries; new code should never reference it.
Angular 19+	The deprecated <code>ComponentFactoryResolver / NgModuleFactory</code> -based creation path is removed from being the recommended or documented API; modern app code has no legitimate reason to import it. (Some low-level APIs retain internal factory concepts for backwards-compatible library interop, but userland code exclusively uses <code>createComponent()</code> .)

Interview framing: "we used to need a resolver + factory + `entryComponents` because View Engine's compiler only generated instantiation code for components it saw referenced somewhere (a template or an explicit `entryComponents` list); Ivy's locality principle means every component compiles its own self-sufficient definition, so instantiation-by-class-reference works everywhere, and the factory/resolver layer became pure ceremony that was eventually deleted."

5. Edge Cases & Gotchas

5.1 Forgetting `.destroy()` → memory leaks

`viewContainerRef.createComponent()` does **not** automatically destroy the component when, e.g., the variable holding the `ComponentRef` goes out of scope, or when some unrelated `*ngIf` toggles off elsewhere in the app. The component instance, its subscriptions, its DOM node, and its place in the view tree persist until something explicitly removes it:

- `viewContainerRef.clear()` — destroys all views in that container.
- `viewContainerRef.remove(index)` — destroys one.
- `componentRef.destroy()` — destroys that specific component's view directly, regardless of which container (if any) holds it.

Leak scenario: a toast service creates a new `ComponentRef` per notification but only ever calls `vcr.createComponent(...)`, never `.destroy()` on the returned ref after the toast should disappear (e.g., after a `setTimeout`). Each toast leaves behind:

- A live component instance (and everything it retains — subscriptions to services, timers, DOM event listeners).
- A detached-but-still-referenced `LView` sitting inside the container's internal view array — increasingly large, growing `viewContainerRef.length`, degrading change-detection performance over the session because Angular walks every view in that container on every tick.
- If the component itself subscribed to a long-lived service `Observable` (e.g., `interval()` or a store) without unsubscribing in `ngOnDestroy` — except `ngOnDestroy` never even runs — that subscription runs forever, doing work and holding closures alive.

Fix: always pair every `createComponent` with a corresponding `destroy()` (or a `clear()` / `remove()` on its container) on whatever event ends that component's life, and always implement `ngOnDestroy` on the dynamic component itself to clean up subscriptions/timers, same as any other component — dynamic creation does not exempt it from normal `onDestroy` discipline.

5.2 Injector context surprises

- If you pass no custom `injector`, the dynamic component's parent injector is the `viewContainerRef`'s injector — which is usually fine, but if the `viewContainerRef` came from an anchor deep inside a component with `providers: [...]` at the component level, the dynamic component *will* see those providers too (they're ordinary ancestors in the injector chain) — this can be a surprise source of "why does my dynamically created component get a different service instance than I expected" bugs.
- For components created via the **free** `createComponent()` **function** (no container), you must supply *both* an `environmentInjector` (resolves module-level/root providers, pipes, etc.) and typically an `elementInjector` (parent for the component's own `NodeInjector`, for anything `providedIn: 'root'` — this is less critical, but for scoped/per-instance tokens like `MODAL_DATA` it's mandatory) — omitting `environmentInjector` throws `NG0400`-class errors about missing providers or fails silently to resolve tokens that only exist in the module tree.
- `inject()` calls inside a dynamically created component's constructor work exactly as normal — DI

resolution doesn't care *how* the component was instantiated, only what injector chain it ends up wired to.

5.3 Manual output subscription instead of template syntax

In a template, `(save)="onSave($event)"` is compiler-generated glue. For a dynamically created component you must wire outputs yourself:

```
const ref = this.vcr.createComponent(EditorComponent);
const sub = ref.instance.save.subscribe((value) => this.onSave(value));

// You now own that subscription's lifecycle manually:
ref.onDestroy(() => sub.unsubscribe());
```

Gotchas:

- If `save` is an `EventEmitter`, subscribing directly on `ref.instance.save` works, but forgetting to `unsubscribe()` when the component is destroyed leaks the subscription (the `EventEmitter / Subject` itself may be garbage collected with the instance, but only once *all* its subscribers are gone — if your subscriber closure captures `this` from an outer long-lived service, that service keeps the destroyed component's emitter, and transitively its last-emitted values/closures, alive).
- `ref.onDestroy(callback)` is the correct place to put this cleanup — it fires exactly when `.destroy()` runs (or when the container that holds it is cleared), mirroring what `ngOnDestroy` would give you inside the component itself.
- With **signal-based outputs** (`output()`, Angular 17.3+) the same idea applies: `ref.instance.save.subscribe(...)` — signal outputs still expose an `OutputRef / Observable`-like `.subscribe()`, so the manual wiring pattern is unchanged.

5.4 Change detection not running automatically for dynamically set inputs

- Setting `ref.instance.someInput = value` directly **bypasses** Angular's input-change bookkeeping: `ngOnChanges` will not fire for that property, and if the component uses `ChangeDetectionStrategy.OnPush`, the component may not even be marked dirty for the next check, so the new value never renders until *something else* happens to mark that view dirty (e.g., an unrelated event within its zone).
- `ref.setInput('someInput', value)` (v14+) is the fix — it goes through Angular's `NgOnChangesFeature` /dirty-marking machinery exactly as a template binding would, correctly triggering `ngOnChanges`, marking `OnPush` views dirty, and working uniformly whether the target declares a decorator `@Input()` or a signal `input()`.
- Even with `setInput`, if the dynamic component's host view was never attached to a tracked tree (free-function `createComponent` without `ApplicationRef.attachView`), no amount of correct dirty-marking will cause a *re-render*, because nothing ever calls `tick()` on that view. Symptom: input updates "work" (no errors, `ngOnChanges` fires) but the DOM visibly never updates — always check `attachView` first when debugging this.
- In zoneless apps (`provideExperimentalZonelessChangeDetection()` / stable zoneless as of Angular 20), the same `setInput / attachView` requirements apply, but there's no `zone.js` patch triggering

`tick()` automatically after arbitrary async work — you rely on Angular's signal-based/notification-based scheduling, which `setInput` correctly participates in (it internally schedules a CD pass), whereas raw `instance.prop = x` assignment does not notify the scheduler at all.

5.5 Other common pitfalls

- **Order of creation vs. DOM position:** `createComponent(Type, { index })` controls the position *inside the container's own view list*, not absolute DOM position relative to siblings outside the container — easy to misplace elements when mixing multiple containers.
- **Re-creating instead of updating:** naively calling `vcr.clear()` + `createComponent()` on every data change (as in a naive dynamic-forms implementation) destroys and rebuilds every field on every keystroke-adjacent change if `ngOnChanges` isn't scoped carefully — prefer diffing the schema and only creating/destroying components whose *type* actually changed, updating inputs via `setInput` for the rest.
- **Content projection into dynamic components:** `projectableNodes` must be actual `Node[][]`, positionally aligned to the target component's `ngContentSelectors` (visible via `componentRef.componentType`'s definition) — passing raw Angular templates instead of DOM nodes is a common mistake; you must render/extract DOM nodes first (often via a hidden `createEmbeddedView` + reading `.rootNodes`).
- **NgComponentOutlet identity churn:** passing a new object literal to `inputs` on every parent change detection (`[NgComponentOutletInputs]="{ x: 1 }"` inline) causes Angular to treat it as changed input state each cycle in older 17.1 behavior nuances — prefer a stable, memoized object reference.

6. Interview Questions & Answers

Q1. What's the difference between a component declared in a template and one created dynamically via `ViewContainerRef`?

A template-declared component is compiled into the parent's view definition at build time — the compiler emits the instantiation instructions directly. A dynamically created component is instantiated at runtime by calling `viewContainerRef.createComponent(SomeComponent)`, with no reference to it anywhere in any template; the decision of *which* component class to instantiate, and *whether* to instantiate it at all, is made by imperative TypeScript logic, often driven by runtime data.

Q2. How do you set an `@Input()` on a dynamically created component, and why is `ComponentRef.setInput()` preferred over `ref.instance.prop = value`?

Interviewer intent: checking whether the candidate actually understands Angular's change-detection dirty-marking, not just that an API exists.

`ref.setInput('prop', value)` (Angular 14+) is preferred because direct property assignment (`ref.instance.prop = value`) bypasses Angular's input bookkeeping entirely: `ngOnChanges` won't fire, and on an `OnPush` component the view may not get marked dirty for the next check, so the new value can silently fail to render until something unrelated triggers CD. `setInput` goes through the same machinery a template binding would — it records the previous/current value pair for `ngOnChanges`, and marks the component's view dirty so `OnPush` components refresh correctly. It's also required for compatibility with signal-based `input()`s.

Q3. How do you subscribe to a dynamically created component's `@Output()`?

There's no template to write `(event)="handler($event)"`, so you subscribe manually on the `ComponentRef`: `const sub = ref.instance.save.subscribe(v => this.onSave(v));`. Because there's no template teardown to auto-unsubscribe this, you must unsubscribe yourself, typically inside `ref.onDestroy(() => sub.unsubscribe())` so it's cleaned up in lockstep with the component's own destruction.

Q4. Why does `ViewContainerRef.createComponent()` no longer require a `ComponentFactoryResolver`?

Prior to Ivy, Angular's View Engine compiler only generated a `ComponentFactory` for components that were referenced in a template or explicitly listed in an `NgModule`'s `entryComponents`; you had to ask a `ComponentFactoryResolver` to look that factory up by type. Ivy compiles every component with a self-contained definition (`@cmp`) directly on the class (the "locality principle"), so any component class can be instantiated purely by reference — no lookup step, no `entryComponents` declaration, no factory object. Angular 13 added the `createComponent(Type, options)` overload that takes the class directly, and `ComponentFactoryResolver` was deprecated, then removed from the recommended/public API in later majors.

Q5. What happens if you create a component via the free `createComponent()` function from `@angular/core` but never call `ApplicationRef.attachView()`?

Interviewer intent: distinguishing candidates who've only used `ViewContainerRef.createComponent` (auto-attached) from those who've built overlay/portal-style code with the standalone function.

The component's `hostView` is a fully detached `ViewRef` that isn't reachable from `ApplicationRef`'s root view list. Angular's change detection walks from tracked roots downward, so a detached view is simply never checked — it may render once during initial creation in some flows, but subsequent input/state changes will not be reflected in the DOM because `tick()` never visits it. You must explicitly call `this.appRef.attachView(componentRef.hostView)` to register it, and correspondingly `detachView()` when done, or the reference lingers inside `ApplicationRef` indefinitely (a leak in the opposite direction — attached-but-abandoned).

Q6. Your team built a toast/notification service using `ViewContainerRef.createComponent()` per message, and after a long session the app becomes sluggish. What's the likely cause and how do you verify it?

Likely cause: toast components are created but never `.destroy()`-ed (or their container is never `.remove()` / `.clear()`-ed) after they're supposed to disappear, so each one remains a live view in the container's internal list — every one of them gets checked on every `ApplicationRef.tick()`, and any subscriptions/timers inside them keep running. To verify, check `viewContainerRef.length` over time (it should return to a small/zero baseline between toasts, not monotonically grow), and check whether `ngOnDestroy` in the toast component ever actually logs/fires — if it never fires, `destroy()` is never being called anywhere.

Q7. How would you pass per-instance configuration data into a dynamically created component without hardcoding it as an `@Input()`?

Use a custom `Injector` with a scoped `InjectionToken`: `Injector.create({ parent: someParentInjector, providers: [{ provide: MY_DATA, useValue: data }] })`, then pass it as the `injector` option to `createComponent`. The dynamic component then does `data = inject(MY_DATA)`. This is exactly how a generic modal/dialog service passes per-open-call data into content components it has never seen at compile time (see `MODAL_DATA` in this chapter's modal service example) — it decouples the target component from any fixed `@Input()` contract the service would otherwise have to know about.

Q8. What's the difference between `NgComponentOutlet` and manually calling

`viewContainerRef.createComponent()`?

`NgComponentOutlet` is a structural directive you use *inside a template* — you bind a `Type<any>` expression to `*ngComponentOutlet` and Angular handles creating/destroying the component for you as that expression changes, plus (17.1+) an `inputs` map it re-applies via `setInput` each cycle. Manual `viewContainerRef.createComponent()` is the imperative escape hatch you use from a service or component class when there's no natural template location to declare the outlet — e.g., a modal service with no fixed host template, or a case needing fine imperative control over exactly when creation/destruction happens (not just "whenever this template re-evaluates").

Q9. Explain how `NgTemplateOutlet`'s context object relates to `let-` template variables.

Interviewer intent: verifying the candidate understands this is the same primitive `*ngFor` / `*ngIf` are built on, not a separate feature.

`*ngTemplateOutlet="tpl; context: { $implicit: value, index: i }"` supplies a plain object; inside the referenced `<ng-template>`, `let-x` (no `= "..."`) binds to `context.$implicit`, while `let-y="index"` binds to `context.index`. This is precisely the mechanism structural directives use internally: `*ngFor` calls `viewContainerRef.createEmbeddedView(templateRef, { $implicit: item, index, first, last, ... })` per item. `NgTemplateOutlet` simply exposes that same `createEmbeddedView` + context capability directly to userland templates, which is how you build render-prop-style reusable components (e.g., a generic list component whose row markup the *consumer* supplies via a `TemplateRef` input).

Q10. Why might a dynamically created component fail to resolve a service that works fine when the same component is used statically in a template?

Because the dynamic component's DI parent chain depends entirely on what injector you passed (or defaulted to) at `createComponent` time. If it was created via the free `createComponent()` function without supplying the right `environmentInjector` / `elementInjector`, module- or component-scoped providers the service depends on may simply not be in the resolved chain, producing an `NG0201` ish "No provider for X" error — even though, when used statically inside some component's template, it would have inherited that component's ancestor chain automatically. The fix is to explicitly pass the correct parent injector(s) so the dynamic component's DI chain matches what it would have had if placed in the intended location.

Q11. In an `onPush` dynamic-forms implementation, you call `ref.setInput('control', formControl)` but the field doesn't visually update on the next value change. What else could be wrong?

`setInput` correctly marks the input dirty and triggers `ngOnChanges` / dirty-marking for that one call, but if the *component's own internal logic* mutates state in a way `onPush` can't observe (e.g., mutating an object in place rather than replacing the reference, or an async callback running outside anything that schedules CD), the view still won't refresh. Also check whether the component's host view is actually attached/reachable (§5.4) — `setInput` on a detached view still won't produce visible DOM updates because nothing ever calls `tick()` on it.

Q12. How do you clean up a dynamically created component to avoid a memory leak, comprehensively (not just the obvious `.destroy()` call)?

Full checklist: (1) call `componentRef.destroy()` (or remove it via its container's `remove()` / `clear()`) exactly when its logical lifetime ends; (2) ensure the component's own `ngOnDestroy` actually unsubscribes from any long-lived observables/timers it created — `destroy()` only calls `ngOnDestroy`, it doesn't automatically know what to clean up inside it; (3) unsubscribe from any manual `ref.instance.output.subscribe(...)` wiring you did externally, typically inside `ref.onDestroy(() => sub.unsubscribe())`; (4) if you used the free `createComponent()` function, call `appRef.detachView(ref.hostView)` — `destroy()` alone does not

automatically detach it from `ApplicationRef`'s tracked view list in every code path, so pairing an explicit `onDestroy` hook that detaches is the safe pattern; (5) remove any manually-appended DOM nodes (e.g., a backdrop element) that weren't part of the component's own view.

Q13. Can dynamically created standalone components be lazy-loaded, and how does that relate to plugin architectures?

Yes — because standalone components need no owning `NgModule`, you can `await import('./widget-x.component')` to fetch the component class from a separate lazy chunk at runtime, then call `viewContainerRef.createComponent(module.widgetXComponent)` once the dynamic `import()` resolves. Combined with a string-keyed registry mapping plugin IDs to `() => import(...)` loader functions, this is the standard pattern for plugin/micro-frontend-style Angular apps: the host defines an extension point (a `ViewContainerRef` anchor) and resolves+instantiates whatever plugin component a runtime configuration or remote manifest specifies, without the host's own bundle ever statically importing the plugin's code.

Q14. What's the internal difference between how `ViewContainerRef.createComponent()` and a plain `*ngIf`-guarded static component get checked by change detection?

There is no meaningful internal difference once the dynamic component's view is inserted into a container that's part of a tracked view tree: both end up as an `LView` reachable from some root, and `ApplicationRef.tick()`'s recursive `refreshView()` walk treats them identically — Angular's change detection has no special-case code path for "was this view created dynamically." The only place a real difference can appear is if the dynamic component was created through the free-standing `createComponent()` function with no container and never attached via `ApplicationRef.attachView()` — in that case it's simply not part of any tree Angular walks, which is a difference in *reachability*, not in how CD itself works once reachable.

7. Quick Revision Cheat Sheet

- **Get a container:** `@ViewChild('anchor', { read: ViewContainerRef })`, or inject `ViewContainerRef` directly in a directive/component, or `{ read: ViewContainerRef }` on an `ng-template` ref.
- **Create (modern, v13+):** `vcr.createComponent(SomeComponent, { index?, injector?, projectableNodes? })` — no `ComponentFactoryResolver`, no `entryComponents`.
- **Set inputs:** `ref.setInput('name', value)` (v14+) — NOT `ref.instance.name = value` (skips `ngOnChanges` / `onPush` dirty-marking).
- **Listen to outputs:** `ref.instance.someOutput.subscribe(fn)`; unsubscribe manually, ideally inside `ref.onDestroy(() => sub.unsubscribe())`.
- **Destroy:** `ref.destroy()`, or `vcr.remove(index)` / `vcr.clear()`. Always pair every `createComponent` with a corresponding teardown path — this is the #1 leak source.
- **Unattached creation** (services/overlays with no fixed container): `createComponent(Type, { environmentInjector, elementInjector? })` from `@angular/core`, then `appRef.attachView(ref.hostView)` and manually append `ref.location.nativeElement` to the DOM (commonly `document.body`); remember `appRef.detachView()` on teardown.
- **Custom DI for dynamic components:** `Injector.create({ parent, providers: [...] })`, pass as `injector` / `elementInjector` option — the standard way to hand per-instance data (`MODAL_DATA`-style tokens) to a component the caller doesn't statically know.
- `NgComponentOutlet`: template-level dynamic component rendering — `*ngComponentOutlet="type"`;

`inputs: {...}; injector: inj; content: nodes"`. Best when the outlet location is inside a template you control.

- **NgTemplateOutlet**: renders an `ng-template`'s embedded view with an optional context object (`$implicit`, named keys) — same primitive `*ngFor` / `*ngIf` use internally via `createEmbeddedView`. Best for render-prop-style reusable components.
- **ComponentFactoryResolver**: legacy, View-Engine-era requirement; deprecated Angular 13, gone from the modern/recommended API by Angular 19 — never write new code that uses it.
- **Change detection reachability**: a view only gets checked if it's part of a tree rooted at something `ApplicationRef` tracks. Container-based `createComponent` is auto-attached; free-function `createComponent` is not — you must `attachView` / `detachView` yourself.
- **Common gotchas**: forgetting `.destroy()` (leak), forgetting `appRef.attachView()` (never re-renders / never gets cleaned up), direct `instance.prop = x` assignment on `OnPush` targets (silently stale UI), forgetting to unsubscribe manually-wired outputs, misaligned `projectableNodes` vs. `ngContentSelectors`.

Created By - Durgesh Singh